
UNIVERSIDAD NACIONAL DEL ALTIPLANO

Facultad de Ingeniería Mecánica Eléctrica, Electrónica y Sistemas

Escuela Profesional de Ingeniería de Sistemas

INFORME TÉCNICO

Análisis de complejidad
y benchmark empírico:
búsqueda lineal vs. búsqueda binaria

SIS210 · Algoritmos y Estructuras de Datos · Semana 2 · Complejidad

Estudiantes: Merhi Estefany Quispe Zela

Stefany Marie Pampa Yucra

Angheli Luana Cuevas Yucra

Docente: Aldo Hernán Zanabria Gálvez

Curso: Algoritmos y Estructuras de Datos (SIS210)

Puno — Perú — 2026

Índice

1	Problema y objetivo	2
2	Marco teórico: $T(n)$, O, Ω, Θ y casos	2
2.1	Tamaño de entrada n y operación dominante	2
2.2	Función de costo $T(n)$	2
2.3	Notación asintótica (O , Ω , Θ)	3
2.4	¿Por qué Ω no es sinónimo automático de “mejor caso”?	3
3	Dataset y preparación	3
4	Diseño experimental	4
5	Implementaciones y control de variables	4
6	Resultados teóricos y empíricos	5
6.1	Comparaciones (escenario “ausente”, peor caso real)	5
6.2	Tiempo de ejecución (escenario “ausente”, promedio de 30 repeticiones) . .	5
6.3	Gráficos	6
7	Discusión de discrepancias y limitaciones	7
7.1	Desafío avanzado: costo total de muchas consultas	8
8	Conclusiones	8
9	Declaración de uso de IA	9
10	Referencias (APA 7)	9
	Anexo: Preguntas de análisis e investigación	10

1. Problema y objetivo

Una organización de comercio electrónico necesita localizar registros de clientes (identificados por CustomerID) repetidamente dentro de su base de transacciones. Este informe compara dos estrategias de búsqueda —lineal y binaria— desde el punto de vista teórico (complejidad asintótica) y empírico (mediciones reales de comparaciones y tiempo), usando como caso de estudio el dataset real Online Retail (UCI Machine Learning Repository).

Objetivo: derivar las cotas asintóticas de ambos algoritmos, implementarlos de forma comparable en Python 3 y C++20, y contrastar la predicción teórica con evidencia empírica reproducible, siguiendo el procedimiento y los requisitos de la guía de práctica de laboratorio SIS210 — Semana 2.

2. Marco teórico: $T(n)$, O , Ω , Θ y casos

2.1. Tamaño de entrada n y operación dominante

n = número de elementos del arreglo/lista donde se realiza la búsqueda. La operación dominante es la comparación entre el valor buscado y un elemento del arreglo (`if valor == x` en la lineal; `if a[medio] == x` en la binaria), que es la que se cuenta en la variable `ops` del código base y la que determina el costo real de cada algoritmo.

2.2. Función de costo $T(n)$

- Búsqueda lineal — peor caso (valor al final o ausente): $T(n) = n$
- Búsqueda lineal — mejor caso (valor en la posición 0): $T(n) = 1$
- Búsqueda lineal — caso promedio: $T(n) \approx n/2$
- Búsqueda binaria — peor caso (valor ausente o en un extremo): $T(n) \approx \lfloor \log_2(n) \rfloor + 1$
- Búsqueda binaria — mejor caso (valor justo en la posición medio): $T(n) = 1$
- Búsqueda binaria — caso promedio: $T(n) \approx \log_2(n)$

Justificación del logaritmo en la búsqueda binaria: cada iteración descarta la mitad del espacio de búsqueda restante. Partiendo de n elementos, tras k iteraciones quedan $n/2^k$ elementos. El proceso termina cuando $n/2^k = 1$, es decir, $k = \log_2(n)$.

2.3. Notación asintótica (O , Ω , Θ)

$O(g(n))$: cota superior. $T(n) \in O(g(n))$ si existen constantes $c > 0$ y n_0 tales que $T(n) \leq c \cdot g(n)$ para todo $n \geq n_0$.

$\Omega(g(n))$: cota inferior. $T(n) \in \Omega(g(n))$ si existen $c > 0$ y n_0 tales que $T(n) \geq c \cdot g(n)$ para todo $n \geq n_0$.

$\Theta(g(n))$: cota ajustada, cuando $T(n)$ está simultáneamente en $O(g(n))$ y en $\Omega(g(n))$.

Estas tres notaciones describen la forma de crecimiento de $T(n)$, no un escenario de entrada. “Mejor caso” y “peor caso” describen qué tipo de entrada recibe el algoritmo; son dos ejes independientes que se combinan (por ejemplo, “el peor caso de la lineal es $\Theta(n)$ ”), pero no son sinónimos entre sí.

Algoritmo	Mejor caso	Peor caso	Caso promedio
Lineal	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Binaria	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$

2.4. ¿Por qué Ω no es sinónimo automático de “mejor caso”?

En el peor caso de la búsqueda lineal, $T(n) = n$. Ese mismo peor caso tiene una cota superior $O(n)$ y también una cota inferior $\Omega(n)$ (en el peor caso, el algoritmo nunca hace menos de n comparaciones). Es decir, Ω también describe el peor caso, no solo el mejor. Ω únicamente expresa “esto no puede crecer más lento que...”, sin importar de qué caso se trate.

3. Dataset y preparación

Dataset: Online Retail, UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5BW33>. Licencia: CC BY 4.0. Periodo: 01/12/2010–09/12/2011. Instancias originales: 541 909 transacciones.

Clave elegida: **CustomerID**, siguiendo el escenario propuesto por la propia guía de práctica (“localizar CustomerID repetidamente”). Tras eliminar los valores nulos de esta columna, quedaron 406 829 filas válidas (de 541 909 totales), con 4 372 valores de **CustomerID** únicos.

Limitación documentada: la guía de práctica sugiere tamaños de muestra $n = 100$, 1 000, 10 000, 100 000 y 500 000 “si el equipo lo permite”. En este caso, el límite no fue de hardware sino de datos disponibles: el dataset, tras la limpieza de nulos, solo contiene

406 829 filas válidas — menos que 500 000. Por esta razón, el experimento se ejecutó con $n = 100, 1\,000, 10\,000$ y $100\,000$, sin inventar ni extrapolar datos para alcanzar el tamaño mayor.

Para cada n , se tomó una muestra aleatoria sin repetición de `CustomerID` reales, usando una semilla fija (`seed=42`) para garantizar reproducibilidad exacta. La búsqueda lineal utiliza el arreglo en el orden real de aparición en el dataset; la búsqueda binaria utiliza una copia ordenada de la misma muestra. El tiempo de ordenar esa copia se midió y reportó por separado del tiempo de búsqueda, tal como exige la guía de práctica.

4. Diseño experimental

- Tratamiento: búsqueda lineal vs. búsqueda binaria.
- Tamaños de muestra: $n = 100; 1\,000; 10\,000; 100\,000$.
- Escenarios: objetivo al inicio, centro, final y ausente (definidos sobre la posición en el arreglo ordenado, usando la misma clave objetivo para ambos algoritmos).
- Repeticiones: 30 por cada combinación de $n \times$ escenario $\times$ algoritmo.
- Variables registradas: número de comparaciones y tiempo en nanosegundos (`perf_counter_ns`).
- Estadísticos calculados: promedio, mediana, mínimo y máximo.
- Control: misma muestra de datos reales para ambos algoritmos; mismo objetivo de búsqueda; tiempo de ordenamiento separado del tiempo de búsqueda.

Total de mediciones de búsqueda: $4 \text{ tamaños} \times 4 \text{ escenarios} \times 30 \text{ repeticiones} \times 2 \text{ algoritmos} = 960$ filas de datos crudos.

5. Implementaciones y control de variables

Se implementaron versiones comparables de ambos algoritmos en Python 3 y C++20, a partir del código base provisto por la guía de práctica, sin modificar la lógica interna de las funciones. La verificación de línea base (Fase A) confirmó que, para la misma entrada (arreglo $0 \dots 19$, objetivo=15), ambas implementaciones producen la misma posición encontrada y el mismo número de comparaciones (lineal: 16 comparaciones; binaria: 4 comparaciones), validando que las implementaciones son comparables entre lenguajes.

Nota sobre el entorno de ejecución: durante el desarrollo se detectó un error de compilación (“`undefined reference to WinMain`”) en dos instalaciones distintas de g++ en

Windows (MinGW-w64 y la distribución incluida con Qt), lo cual impidió compilar y ejecutar C++ de forma local de manera confiable. Como solución, la verificación del código C++ y el benchmark completo con datos reales se ejecutaron en Google Colab (entorno Linux), donde el mismo código compiló y corrió sin inconvenientes. Esto no afecta la validez del experimento, ya que el benchmark cuantitativo completo (960 mediciones) se realizó en Python, y la comparabilidad de la lógica entre Python y C++ ya había sido validada en la Fase A.

El benchmark completo (instrumentación, muestreo, medición y generación de gráficos) se consolidó en un único script reproducible (`benchmark.py`), que produce resultados idénticos en ejecuciones repetidas gracias a la semilla fija utilizada para el muestreo.

6. Resultados teóricos y empíricos

La predicción teórica establece que la búsqueda lineal crece de forma lineal ($\Theta(n)$) y la búsqueda binaria de forma logarítmica ($\Theta(\log n)$) en el peor caso. Los resultados empíricos, obtenidos con 30 repeticiones por combinación sobre datos reales de `CustomerID`, confirman esta predicción con alta fidelidad.

6.1. Comparaciones (escenario “ausente”, peor caso real)

n	Comparaciones — Lineal	Comparaciones — Binaria
100	100	7
1 000	1 000	10
10 000	10 000	14
100 000	100 000	17

La búsqueda lineal realiza exactamente n comparaciones en todos los tamaños, como predice la teoría. La búsqueda binaria crece de forma logarítmica (7, 10, 14, 17), muy cercana al valor teórico $\lceil \log_2(n) \rceil + 1$.

6.2. Tiempo de ejecución (escenario “ausente”, promedio de 30 repeticiones)

La brecha de tiempo entre ambos algoritmos crece junto con n : en $n = 100$ la lineal es poco menos de 2 veces más lenta que la binaria; en $n = 100\,000$, es aproximadamente 5 veces más lenta. Esto confirma que la ventaja de la búsqueda binaria se vuelve más pronunciada a medida que n crece, en línea con la diferencia entre $\Theta(n)$ y $\Theta(\log n)$.

n	Tiempo — Lineal (ns)	Tiempo — Binaria (ns)
100	7 919,87	4 308,03
1 000	96 901,07	23 299,67
10 000	1 191 732,47	232 491,60
100 000	20 811 188,20	4 194 507,20

6.3. Gráficos

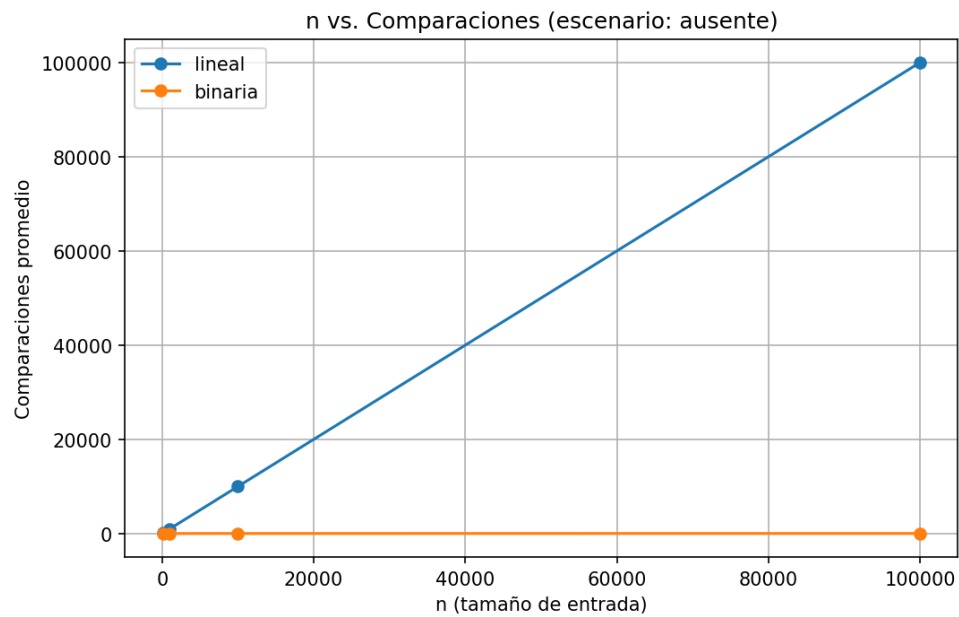


Figura 1: n vs. Comparaciones promedio (escenario ausente).

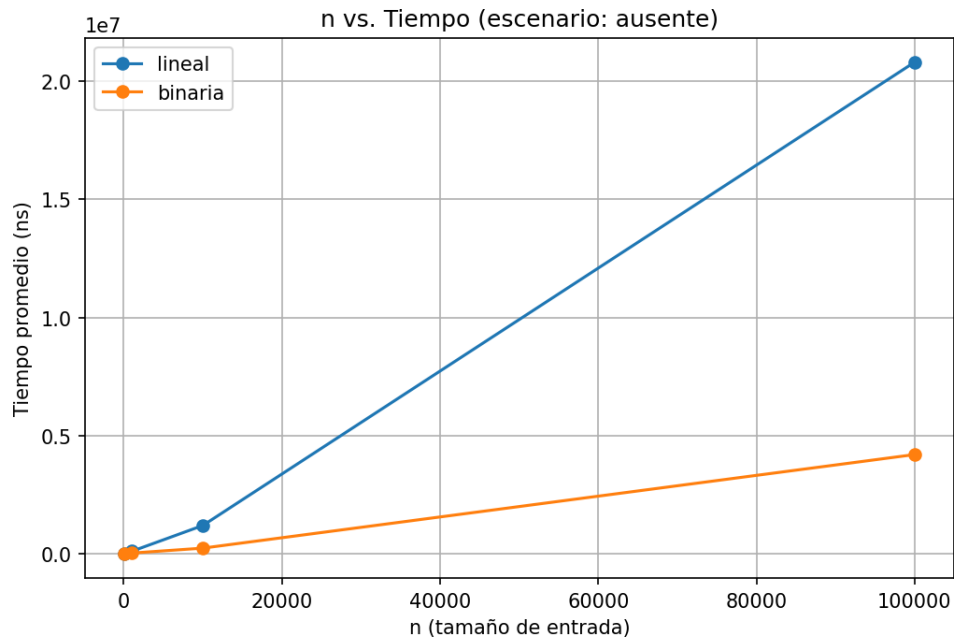


Figura 2: n vs. Tiempo promedio en ns (escenario ausente).

7. Discusión de discrepancias y limitaciones

Aunque las comparaciones siguen la teoría casi exactamente, los tiempos muestran variabilidad que la complejidad asintótica no predice, porque esta describe la forma de crecimiento y no la velocidad real. Factores que explican esta diferencia:

- Constantes ocultas por operación: cada comparación tiene un costo real distinto (acceso a memoria, evaluación de la condición, control del bucle), que la notación asintótica ignora deliberadamente.
- Efectos de caché y localidad de memoria: la búsqueda binaria accede a posiciones dispersas del arreglo, mientras que la lineal accede de forma secuencial; ambos patrones interactúan de forma distinta con la caché del procesador.
- Entorno de ejecución compartido: el experimento se corrió en Google Colab, un entorno virtualizado en la nube que puede compartir recursos con otros procesos, introduciendo variabilidad que no ocurriría en una máquina dedicada.
- Limitación del dataset real: el tamaño máximo alcanzado fue $n = 100\,000$ en lugar de $500\,000$, porque la columna `CustomerID` solo contiene $406\,829$ filas con valores no nulos (de $541\,909$ totales) y $4\,372$ valores únicos.

Estos hallazgos distinguen claramente complejidad (comparaciones, independiente del hardware) de velocidad real (tiempo, dependiente del hardware y el entorno).

7.1. Desafío avanzado: costo total de muchas consultas

Usando el modelo $\text{costo_total} = \text{preparación} + q \times \text{costo_por_consulta}$, con datos reales de $n = 100\,000$ (escenario “ausente”):

- Escenario A (sin ordenar, lineal repetida): $\text{costo_A}(q) = q \times 20\,811\,188, 20 \text{ ns}$
- Escenario B (ordenar una vez, binaria repetida): $\text{costo_B}(q) = 1\,680\,260 \text{ ns} + q \times 4\,194\,507, 20 \text{ ns}$

Punto de equilibrio: $q \approx 0,10$ consultas.

Esto significa que, en esta implementación concreta, ordenar y usar búsqueda binaria conviene desde la primera consulta, no solo cuando hay “miles de consultas” como sugiere la intuición teórica clásica. La razón es que el ordenamiento (`np.sort`, implementado en C internamente por NumPy) es mucho más rápido que recorrer el arreglo completo con un bucle `for` puro de Python. Esto ilustra que la complejidad asintótica (que compara $O(n \log n)$ vs. $O(n)$ en el papel) no predice el costo real cuando las constantes de cada implementación son muy distintas entre sí. En un caso con constantes más parejas entre ambos algoritmos, el punto de equilibrio sería mayor, y el modelo clásico se cumpliría de forma más clara.

Estructura que evitaría esta decisión: una tabla hash (`dict` en Python, `unordered_map` en C++) permite búsqueda en tiempo promedio $O(1)$, sin necesitar que los datos estén ordenados. Esto eliminaría el dilema de “ordenar o no ordenar”, a costa de mayor uso de memoria y de perder el orden de los elementos (útil si en el futuro se necesitaran consultas por rango).

8. Conclusiones

- Las cotas asintóticas derivadas teóricamente ($\Theta(n)$ para la lineal, $\Theta(\log n)$ para la binaria) se confirmaron empíricamente con alta fidelidad en el número de comparaciones.
- El tiempo real, aunque sigue la misma tendencia general, presenta mayor variabilidad que las comparaciones puras, debido a factores de hardware, lenguaje y entorno de ejecución no capturados por la notación asintótica.

- La ventaja de la búsqueda binaria sobre la lineal crece junto con n , tanto en comparaciones como en tiempo, validando su preferencia para colecciones grandes y estables.
- El desafío avanzado mostró que la decisión de “ordenar o no ordenar” depende fuertemente de las constantes reales de implementación, y no únicamente de la complejidad teórica en el papel.
- Se documentó explícitamente la limitación del tamaño máximo de muestra (100 000 en vez de 500 000) debido a la disponibilidad real de datos, sin recurrir a datos inventados o extrapolados.

9. Declaración de uso de IA

Durante el desarrollo de esta práctica se utilizó Claude (Anthropic) como asistente de apoyo para: comprender y explicar los conceptos teóricos de complejidad (O , Ω , Θ , casos), guiar paso a paso la configuración del entorno de desarrollo (VS Code, Git, Colab) y el diagnóstico de un error de compilador en C++, redactar y organizar el código de instrumentación y benchmark a partir de las especificaciones de la guía de práctica, ayudar a estructurar y redactar este informe técnico, y revisar el trabajo contra la rúbrica de evaluación.

Todos los resultados numéricos, gráficos y CSV presentados en este informe provienen de la ejecución real del código sobre el dataset Online Retail en el entorno de la estudiante (Google Colab); no se generaron ni simulaban resultados de forma artificial. Las decisiones de diseño del experimento (elección de la clave `CustomerID`, ajuste del tamaño máximo de muestra ante la limitación real de datos, interpretación de la portada del PDF respecto a C++17/C++20) fueron discutidas y aprobadas explícitamente por la estudiante antes de implementarse.

10. Referencias (APA 7)

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press. <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>
- Black, P. E. (2020). *DADS: The on-line dictionary of algorithms and data structures*. National Institute of Standards and Technology. <https://www.nist.gov/publications/dads-line-dictionary-algorithms-and-data-structures>

- Python Software Foundation. (2026). *time* — *Time access and conversions*. Python 3.14.7 documentation. <https://docs.python.org/3/library/time.html>
- cppreference.com. (2026). *Date and time utilities*. <https://en.cppreference.com/w/cpp/chrono>
- Chen, D. (2015). *Online Retail* [Data set]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5BW33>
- Skiena, S. S. (2020). *The algorithm design manual* (3rd ed.). Springer. <https://link.springer.com/book/10.1007/978-3-030-54256-6>

Anexo: Preguntas de análisis e investigación

1. ¿Puede un algoritmo $\Theta(n)$ ser más rápido que uno $\Theta(\log n)$ para ciertos n ? Explique el papel de las constantes.

Sí, para valores de n pequeños. La notación asintótica solo describe el comportamiento cuando n es lo suficientemente grande ($n \geq n_0$); para n chicos, las constantes ocultas en cada algoritmo pueden hacer que el de “peor” complejidad gane. Un ejemplo real de esto aparece en el desafío avanzado de este mismo informe: ordenar con `np.sort()` (una operación $O(n \log n)$) fue más rápido en la práctica que recorrer el arreglo con búsqueda lineal ($O(n)$) para una sola consulta, porque la constante de NumPy en C es muchísimo menor que la del bucle interpretado de Python.

2. ¿Por qué Ω no es sinónimo automático de mejor caso?

Ω describe una cota inferior de la función de costo (algo que crece “al menos así de rápido”), sin importar de qué escenario de entrada se trate. El peor caso de la búsqueda lineal ($T(n) = n$) tiene tanto una cota superior $O(n)$ como una cota inferior $\Omega(n)$: es decir, Ω también puede describir el peor caso, no solo el mejor.

3. ¿Qué efecto tiene la caché sobre un benchmark aunque no cambie la complejidad?

La caché del procesador es más rápida que la RAM, pero solo guarda datos accedidos recientemente o cercanos en memoria. Un algoritmo que accede de forma secuencial

(búsqueda lineal) aprovecha mejor la caché que uno que salta entre posiciones lejanas (búsqueda binaria). Esto no cambia el número de operaciones (la complejidad es la misma), pero sí puede afectar el tiempo real medido.

4. ¿Cuándo el costo de ordenar invalida una comparación simplista con búsqueda binaria?

Cuando se hace una sola consulta (o muy pocas) sobre datos desordenados que no se van a reutilizar. En ese caso, el costo de ordenar (1 680 260 ns en este experimento, para $n = 100\,000$) se suma una sola vez sin poder amortizarse entre múltiples búsquedas.

5. ¿Cómo cambiaría el experimento si las claves siguieran una distribución no uniforme?

La muestra de este experimento se tomó dando a cada `CustomerID` la misma probabilidad de aparecer. En un dataset real, algunos clientes compran con más frecuencia que otros (en este dataset, 406 829 filas provienen de solo 4 372 clientes únicos). Con una distribución no uniforme, el “caso promedio” ya no sería simplemente $n/2$ comparaciones para la lineal; habría que ponderar cada posición según la probabilidad real de que ese cliente específico sea el buscado.

6. Formule una hipótesis para comparar rendimiento en Python, C++ y una implementación nativa.

Hipótesis: para el mismo algoritmo (misma cantidad de comparaciones), C++ compilado será consistentemente más rápido que Python interpretado, porque Python tiene overhead adicional en cada iteración (verificación de tipos dinámicos, gestión de memoria automática, interpretación bytecode a bytecode) que C++ no tiene al compilar directo a código máquina. Esta hipótesis se verificó parcialmente en la Fase A (línea base, $n = 20$) de este proyecto, pero no se extendió al benchmark completo con n grandes debido al problema de compilador documentado en la sección 5.